FOUNDATION OF JAVA LAB SHEET – 10
FILE HANDLING

Requirements:
VS Code editor or eclipse or IntelliJ Idea IDE
JDK 19 or higher

Q1. Create and Write to a File
Write a Java program to create a file named student.txt and write Student Name, Roll Number, Course, and Semester into it. Display a suitable message a er the file is successfully created and data is written.

Concepts: File, FileWriter, file crea on, wri ng data.

```
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            FileWriter f = new FileWriter("student.txt");

            f.write("Student Name: Sumit\n");
            f.write("Roll Number: 101\n");
            f.write("Course: BCA\n");
            f.write("Semester: 3");

            f.close();

            System.out.println("File created and data written successfully.");
        } catch (IOException e) {
            System.out.println("File error.");
        }
    }
}
```

OUTPUT:
Student Name: Sumit
Roll Number: 101
Course: BCA
Semester: 3

Q2. Read Data from a File Write a Java program to read the contents of student.txt created in Ques on 1 and display the complete contents on the console. If the file does not exist, display an appropriate error message.
Concepts: FileReader, BufferedReader, reading a file, excep on handling.

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(
                    new FileReader("student.txt"));

            String line;
            while ((line = br.readLine()) != null)
                System.out.println(line);

            br.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found.");
        } catch (IOException e) {
            System.out.println("Error reading file.");
        }
    }
}
```

OUTPUT:
Student Name: Sumit
Roll Number: 101
Course: BCA
Semester: 3

Q3. Count Characters, Words and Lines Create a text file named data.txt containing mul ple lines of text. Write a Java program to read the file and calculate the number of lines, words, and characters. Display all three results.
Concepts: File reading, BufferedReader, split(), string processing.

```
import java.io.*;

public class Main {
```

```
    public static void main(String[] args) {
        int lines = 0, words = 0, chars = 0;

        try {
            BufferedReader br = new BufferedReader(
                    new FileReader("data.txt"));

            String line;
            while ((line = br.readLine()) != null) {
                lines++;
                chars += line.length();

                if (!line.trim().isEmpty())
                    words += line.trim().split("\\s+").length;
            }

            br.close();

            System.out.println("Lines: " + lines);
            System.out.println("Words: " + words);
            System.out.println("Characters: " + chars);

        } catch (IOException e) {
            System.out.println("Error reading file.");
        }
    }
}
```

OUTPUT:
Lines: 3
Words: 8
Characters: 38

Q4. Append Data to an Exis ng File Create a Java program that appends new student informa on to an exis ng file named students.txt without dele ng exis ng contents. Accept Student Name, Roll Number, and Marks from the user and add the record at the end.
Concepts: FileWriter, append mode, Scanner.

```
import java.io.FileWriter;
import java.util.Scanner;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            FileWriter f = new FileWriter("students.txt", true);

            System.out.print("Enter Student Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Roll Number: ");
            int roll = sc.nextInt();

            System.out.print("Enter Marks: ");
            double marks = sc.nextDouble();

            f.write("\nName: " + name);
            f.write("\nRoll Number: " + roll);
            f.write("\nMarks: " + marks + "\n");

            f.close();

            System.out.println("Student data appended successfully.");

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:
Enter Student Name: Sumit
Enter Roll Number: 101
Enter Marks: 85.5
Student data appended successfully.

Q5. Copy One File into Another File Write a Java program to copy the contents of source.txt into backup.txt. Display a message a er the copying opera on is completed.
Concepts: FileReader, FileWriter, reading and writing files, character streams.

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("source.txt");
            FileWriter fw = new FileWriter("backup.txt");

            int ch;
            while ((ch = fr.read()) != -1)
                fw.write(ch);

            fr.close();
            fw.close();

            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("Error copying file.");
        }
    }
}
```

OUTPUT:
File copied successfully.

Q6. Search for a Word in a File Create a text file named notes.txt. Write a Java program that accepts a word from the user and searches for that word in the file. Display whether the word is found and the number of mes it occurs.
Concepts: File reading, string searching, contains(), word counting.

```
import java.io.*;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int count = 0;

        System.out.print("Enter word: ");
        String word = sc.nextLine();
```

```
        try {
            BufferedReader br = new BufferedReader(
                    new FileReader("notes.txt"));

            String line;

            while ((line = br.readLine()) != null) {
                String[] words = line.split("\\s+");

                for (String w : words) {
                    w = w.replaceAll("[^a-zA-Z]", "");
                    if (w.equalsIgnoreCase(word))
                        count++;
                }
            }

            br.close();

            if (count > 0)
                System.out.println("Word Found");
            else
                System.out.println("Word Not Found");

            System.out.println("Occurrences: " + count);

        } catch (IOException e) {
            System.out.println("File not found.");
        }
    }
}
```

OUTPUT:
Enter word: Java
Word Found
Occurrences: 3

Q7. Display File Informa on Write a Java program that accepts a file name from the user and displays whether the file exists, file name, absolute path, file size, whether it is a file, whether it is a directory, whether it is readable, and whether it is writable.
Concepts: File class, file proper es and methods.

```
import java.io.File;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter file name: ");
        String name = sc.nextLine();

        File f = new File(name);

        System.out.println("Exists: " + f.exists());

        if (f.exists()) {
            System.out.println("File Name: " + f.getName());
            System.out.println("Absolute Path: " + f.getAbsolutePath());
            System.out.println("File Size: " + f.length() + " bytes");
            System.out.println("Is File: " + f.isFile());
            System.out.println("Is Directory: " + f.isDirectory());
            System.out.println("Readable: " + f.canRead());
            System.out.println("Writable: " + f.canWrite());
        }
    }
}
```

OUTPUT:

```
Enter file name: notes.txt
Exists: true
File Name: notes.txt
Absolute Path: C:\Users\Student\notes.txt
File Size: 120 bytes
Is File: true
Is Directory: false
Readable: true
Writable: true
```

Q8. Count Vowels, Consonants, Digits and Spaces Create a text file named input.txt. Write a Java program to read the file and count the number of vowels, consonants, digits, and spaces. Analyze every character present in the file.

Concepts: File reading, character processing, condi onal statements.

```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        int vowels = 0, consonants = 0, digits = 0, spaces = 0;

        try {
            FileReader fr = new FileReader("input.txt");
            int ch;

            while ((ch = fr.read()) != -1) {
                char c = (char) ch;

                if (Character.isLetter(c)) {
                    if ("AEIOUaeiou".indexOf(c) != -1)
                        vowels++;
                    else
                        consonants++;
                } else if (Character.isDigit(c)) {
                    digits++;
                } else if (c == ' ') {
                    spaces++;
                }
            }

            fr.close();

            System.out.println("Vowels: " + vowels);
            System.out.println("Consonants: " + consonants);
            System.out.println("Digits: " + digits);
            System.out.println("Spaces: " + spaces);

        } catch (IOException e) {
            System.out.println("File error.");
        }
    }
}
```

OUTPUT:
Vowels: 5
Consonants: 12
Digits: 3
Spaces: 2

Q9. Student Record File Write a Java program to accept details of 5 students and store them in student_records.txt. For each student store Roll Number, Name, Course, and Marks. A er storing the records, read the file and display all student records.
Concepts: Scanner, FileWriter, FileReader, BufferedReader, file wri ng and reading.

```
import java.io.*;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            FileWriter fw = new FileWriter("student_records.txt");

            for (int i = 1; i <= 5; i++) {
                System.out.println("Student " + i);

                System.out.print("Roll Number: ");
                int roll = sc.nextInt();
                sc.nextLine();

                System.out.print("Name: ");
                String name = sc.nextLine();

                System.out.print("Course: ");
                String course = sc.nextLine();

                System.out.print("Marks: ");
                double marks = sc.nextDouble();

                fw.write(roll + " | " + name + " | " + course +
                        " | " + marks + "\n");
            }

            fw.close();

            System.out.println("\nStudent Records:");

            BufferedReader br = new BufferedReader(
                    new FileReader("student_records.txt"));

            String line;
```

```
            while ((line = br.readLine()) != null)
                System.out.println(line);

            br.close();

        } catch (IOException e) {
            System.out.println("File error.");
        }
    }
}
```

OUTPUT:
Student Records:
101 | Sumit | BCA | 85.5
102 | Rahul | BCA | 78.0
103 | Aman | BCA | 90.0
104 | Rohit | BCA | 82.5
105 | Karan | BCA | 88.0

Q10. File-Based Employee Management Create a file-based employee management program. Accept Employee ID, Employee Name, Department, and Salary and store them in employees.txt. Then read the file and search for an employee using Employee ID. If found, display the complete record; otherwise display 'Employee record not found.' Handle the situa on where the file does not exist.
Concepts: File creation, writing, reading, searching, File, FileWriter, BufferedReader, exception handling.

```
import java.io.*;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            // Write employee record
            FileWriter fw = new FileWriter("employees.txt");

            System.out.print("Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine();
```

```
        System.out.print("Employee Name: ");
        String name = sc.nextLine();

        System.out.print("Department: ");
        String dept = sc.nextLine();

        System.out.print("Salary: ");
        double salary = sc.nextDouble();

        fw.write(id + "|" + name + "|" + dept + "|" + salary + "\n");
        fw.close();

        // Search employee
        System.out.print("Enter ID to search: ");
        int searchId = sc.nextInt();

        BufferedReader br = new BufferedReader(
                new FileReader("employees.txt"));

        String line;
        boolean found = false;

        while ((line = br.readLine()) != null) {
            String[] data = line.split("\\|");

            if (Integer.parseInt(data[0]) == searchId) {
                System.out.println("Employee Record: " + line);
                found = true;
                break;
            }
        }

        br.close();

        if (!found)
            System.out.println("Employee record not found.");

    } catch (FileNotFoundException e) {
        System.out.println("File does not exist.");
    } catch (IOException e) {
        System.out.println("File error.");
    }
}
```

}

OUTPUT:
Employee ID: 101
Employee Name: Sumit
Department: IT
Salary: 50000
Enter ID to search: 101
Employee Record: 101|Sumit|IT|50000.0